Exp No:1.a) Analyze the trend of data science job postings over the last decade.

Code:

```
import pandas as pd

import matplotlib.pyplot as plt

data = {'Year': list(range(2010, 2021)), 'Job Postings': [150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200]}

 df = pd.DataFrame(data)

 plt.plot(df['Year'], df['Job Postings'], marker='o')

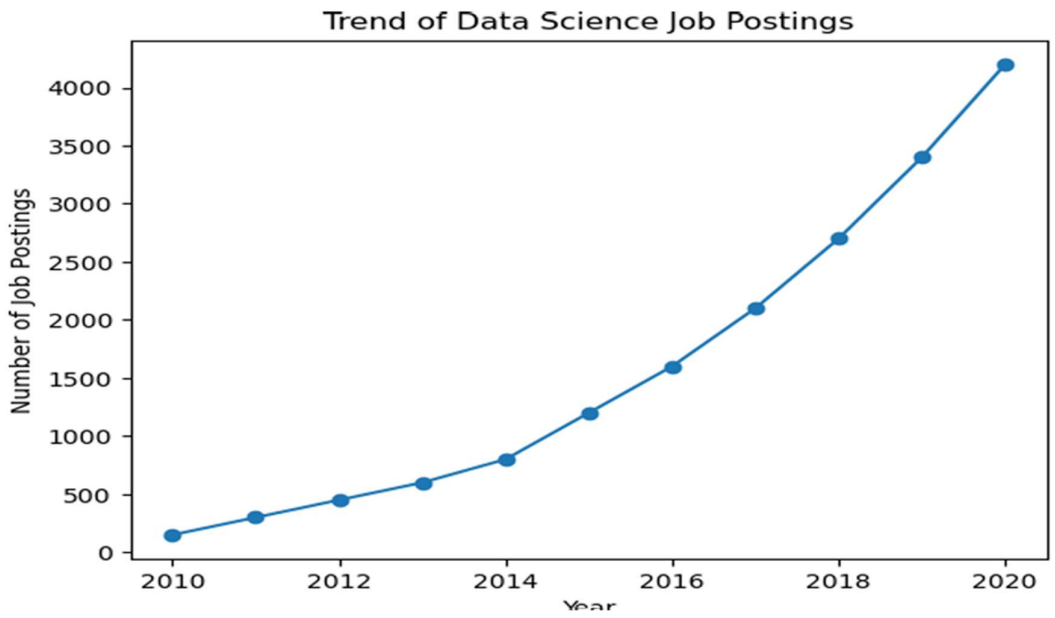
 plt.title('Trend of Data Science Job Postings')

plt.xlabel('Year')

plt.ylabel('Number of Job Postings')

plt.show()
```

Output:



Result:

The trend of data science job postings over the last decade was analysed.

Exp No:1.b) Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

Code:

```
roles = ['Data Analyst', 'Data Engineer', 'Data Scientist', 'ML Engineer', 'Business Analyst']

counts = [300, 500, 450, 200, 150]

plt.bar(roles, counts)

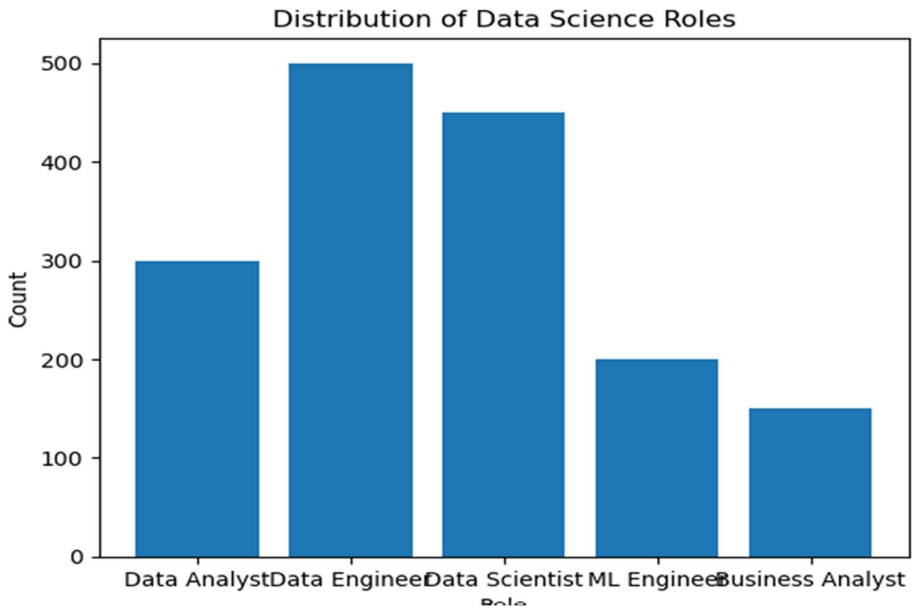
plt.title('Distribution of Data Science Roles')

plt.xlabel('Role')

plt.ylabel('Count')

 plt.show()
```

Output:



Result:

the distribution of various data science roles were analyse and this code was executed successfully.

Exp No:1.c) Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.

Code:

structured_data = pd.DataFrame({ 'ID': [1, 2, 3], 'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35] })

print("Structured Data:\n", structured_data)

unstructured_data = "This is an example of unstructured data. It can be a piece of text, an image, or a video file." print("\nUnstructured Data:\n", unstructured_data)

semi_structured_data = {'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight': 68}}

print("\nSemi-structured Data:\n", semi_structured_data)

Output:

```
Structured Data:
    ID     Name  Age
0   1    Alice   25
1   2      Bob   30
2   3  Charlie   35

Unstructured Data:
 This is an example of unstructured data. It can be a piece of text, an
image, or a video file.

Semi-structured Data:
 {'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight': 68}
}
```

Result:

an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given were analysed.

Exp No:1.d) Conduct an experiment to encrypt and decrypt given sensitive data**.**

Code:

```
from cryptography.fernet import Fernet

key = Fernet.generate_key()

f = Fernet(key) token = f.encrypt(b"Rajalakshmi Engineering College")

 token

 b'...'

 f.decrypt(token)

b'Rajalakshmi Engineering College'

 key = Fernet.generate_key()

cipher_suite = Fernet(key)

plain_text = b"Rajalakshmi Engineering College."

cipher_text = cipher_suite.encrypt(plain_text )

decrypted_text = cipher_suite.decrypt(cipher_text)

print("Original Data:", plain_text)

print("Encrypted Data:", cipher_text)

print("Decrypted Data:", decrypted_text)
```

Output:

```
Original Data: b'Rajalakshmi Engineering College.'
Encrypted Data: b'gAAAAABmmnHEhy1ZC1-Dox-URNAvYqFFGfJemMb_fKmXqhATe3AAn
XEOnSzaNRINI4zAbNK8lgx2pprrNn-j6agbWDf30OwbrZsBuz8fnDdX7N35WwloDu3m6pVv
kwv8GYDSI8Fov1tV'
Decrypted Data: b'Rajalakshmi Engineering College.'
```

Result:

Thus **,** experiment to encrypt and decrypt given sensitive data was analysed and this code executed successfully.

Exp No:2) Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
df = pd.DataFrame({
   'Date': ['01-01-2023', '02-01-2023','03-01-2023','04-01-2023','05-01-2023',],
   'Product': ['Product A', 'Product B','Product A', 'Product C','Product B',],
   'Sales': [200,150,220,300,180,],
   'Quantity': [4,3,5,6,4,],
   'Region': ['North','South','North','East','West',]
})
print(df.head())
print(df.isnull().sum())
df['Sales'] = df['Sales'].fillna(df['Sales'].mean())
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)
print(df.describe())
product_summary = df.groupby('Product').agg({
   'Sales': 'sum',
   'Quantity': 'sum'
}).reset_index()
print(product_summary)
```

```
plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()
df['Date'] = pd.to_datetime(df['Date'])
sales_over_time = df.groupby('Date')['Sales'].sum().reset_index()
plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'], sales_over_time['Sales'],
marker='o')
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.xticks(rotation=45)
plt.show()
pivot_table = df.pivot_table(
    values='Sales',
    index='Region',
    columns='Product',
    aggfunc='sum',
    fill_value=0
)
print(pivot_table)
```

```
correlation_matrix = df.corr(numeric_only=True)

print(correlation_matrix)

plt.figure(figsize=(8, 6))

plt.imshow(correlation_matrix, cmap='coolwarm')

plt.colorbar()

plt.xticks(range(len(correlation_matrix.columns)),
correlation_matrix.columns)

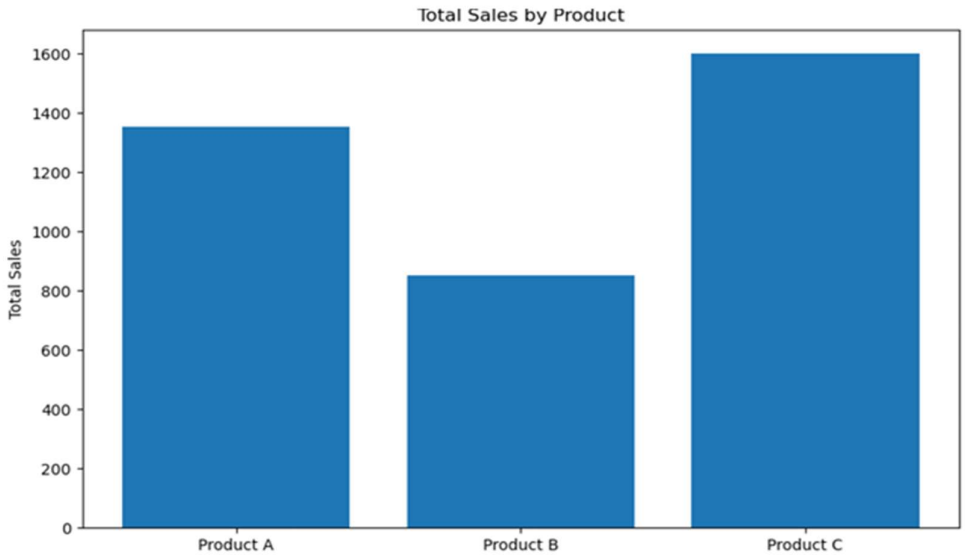
plt.yticks(range(len(correlation_matrix.columns)),
correlation_matrix.columns)

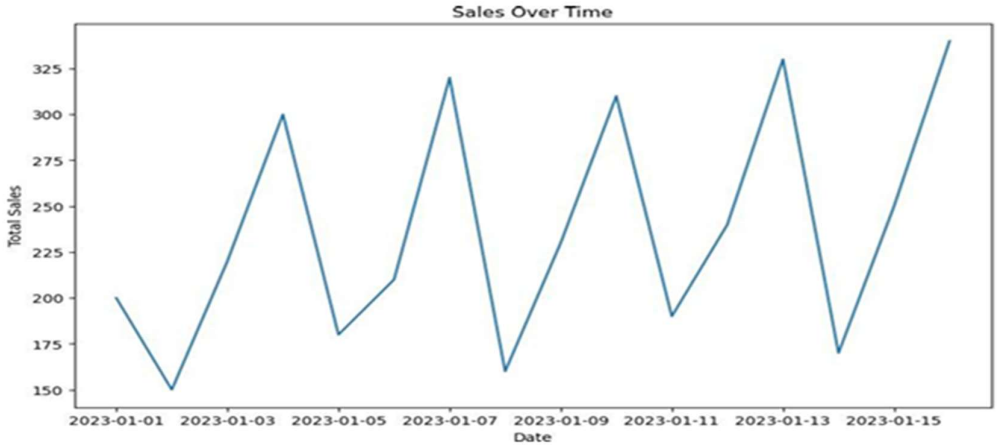
plt.title('Correlation Matrix')

plt.show()
```

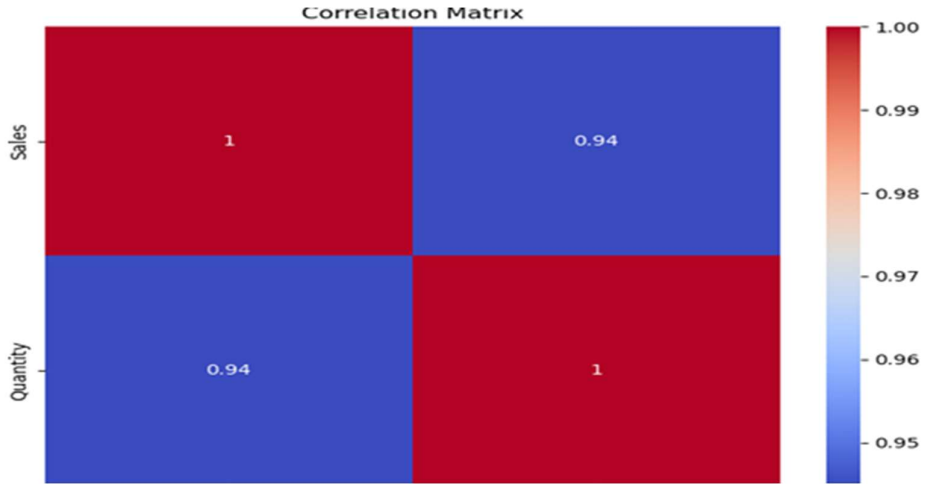
Output:

```
  Date     Product  Sales  Quantity Region
0   2023-01-01  Product A    200         4  North
1   2023-01-02  Product B    150         3  South
2   2023-01-03  Product A    220         5  North
3   2023-01-04  Product C    300         6   East
4   2023-01-05  Product B    180         4   West
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
            Sales   Quantity
count   16.000000  16.000000
mean   237.500000   5.375000
std     64.031242   1.746425
min    150.000000   3.000000
25%    187.500000   4.000000
50%    225.000000   5.500000
75%    302.500000   7.000000
max    340.000000   8.000000
     Product  Sales  Quantity
0  Product A   1350        33
1  Product B    850        17
2  Product C   1600        36
```






```
Product  Product A  Product B  Product C
Region
East             0          0       1600
North         1350          0          0
South            0        480          0
West             0        370          0
             Sales  Quantity
Sales     1.000000  0.944922
Quantity  0.944922  1.000000
C:\Users\Ayyadurai\AppData\Local\Temp\ipykernel_9648\511106317.py:49: Futur
eWarning: The default value of numeric_only in DataFrame.corr is deprecated
. In a future version, it will default to False. Select only valid columns
or specify the value of numeric_only to silence this warning.
  correlation_matrix = df.corr()
```



Result:

The data set given in csv format and perform data preprocessing and visualization was analysed and this code was executed successfully.

Exp No:3) Experiment to understand the data preprocessing in Data science

Code:

```
import numpy as np
import pandas as pd

df = pd.DataFrame({
    'Country': ['India', 'France', 'India', 'France', 'India'],
    'Age': [29, np.nan, 24, 35, 30],
    'Salary': [32000, 40000, 37000, np.nan, 51000],
    'Purchased': ['Yes', 'No', 'Yes', 'No', 'Yes']
})

print("Original Dataset:")
print(df)

print("\n\nDataFrame Information:")
print("\n\nDataFrame Info:")
print(df.info())

print("\n\nCountry Mode:")
print(df['Country'].mode()[0])

print("\n\nFirst Mode Value:")
```

```
print(df['Country'].mode()[0])

df['Country'] = df['Country'].fillna(df['Country'].mode()[0])
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())

print("\n\nDataset After Filling Missing Values:")
print(df)

print("\n\nOne Hot Encoding:")
print(pd.get_dummies(df['Country']))

updated_dataset = pd.concat(
   [pd.get_dummies(df['Country']), df.iloc[:, 1:]],
   axis=1
)

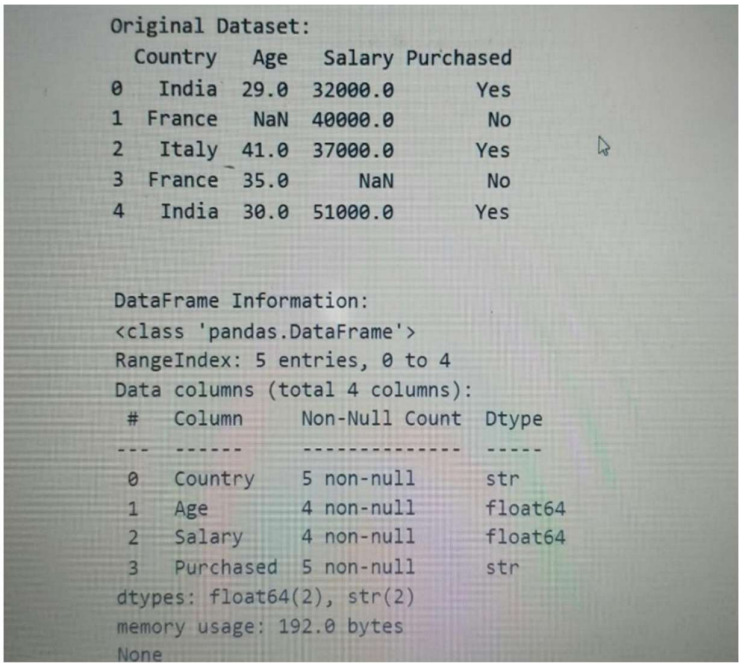
print("\n\nUpdated Dataset:")
print(updated_dataset)

print("\n\nDataFrame Information After Preprocessing:")
print(df.info())
```

```
updated_dataset['Purchased'] =
updated_dataset['Purchased'].replace(
    ['NO', 'Yes'], [0, 1]
)
```

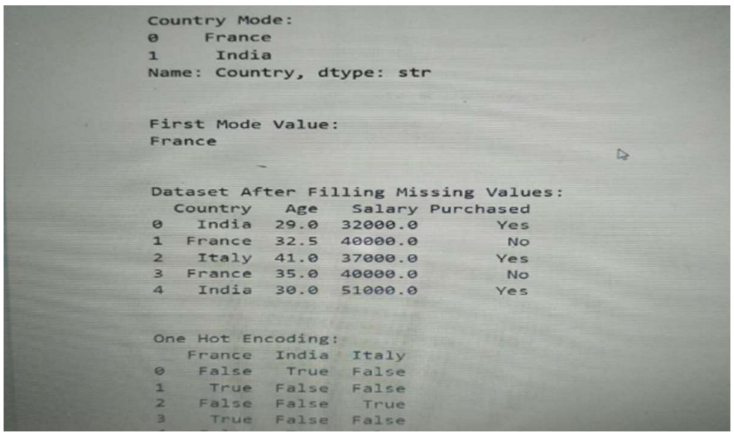
```
print("\n\nFinal Dataset:")
print(updated_dataset)
```

Output:



```
Original Dataset:
  Country   Age   Salary Purchased
0   India  29.0  32000.0       Yes
1  France   NaN  40000.0        No
2   Italy  41.0  37000.0       Yes
3  France  35.0      NaN        No
4   India  30.0  51000.0       Yes

DataFrame Information:
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    5 non-null      str
 1   Age        4 non-null      float64
 2   Salary     4 non-null      float64
 3   Purchased  5 non-null      str
dtypes: float64(2), str(2)
memory usage: 192.0 bytes
None
```

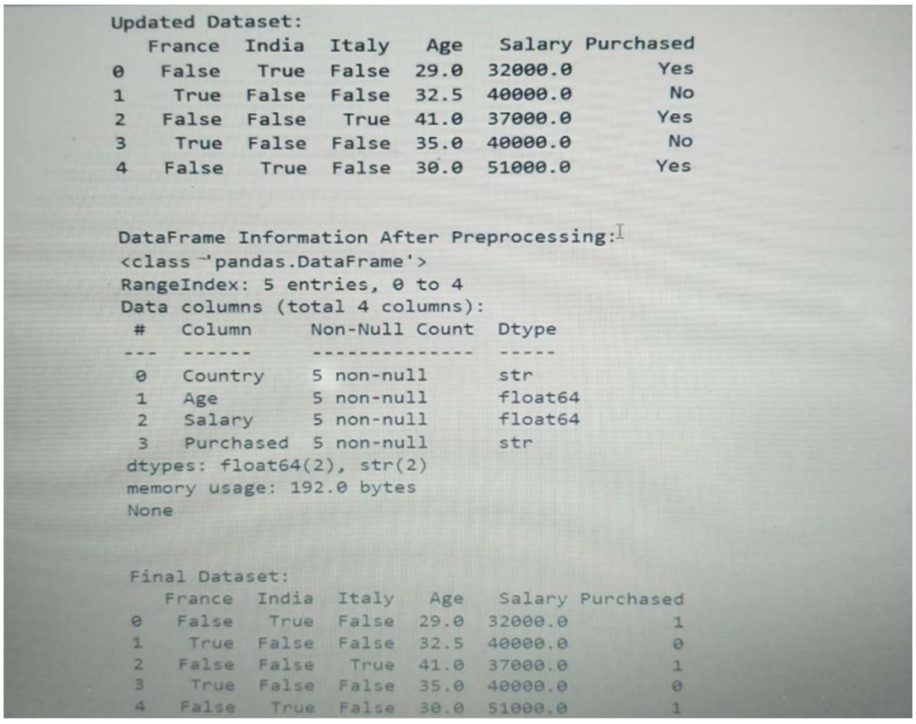


```
Country Mode:
0    France
1     India
Name: Country, dtype: str

First Mode Value:
France

Dataset After Filling Missing Values:
  Country   Age   Salary Purchased
0   India  29.0  32000.0       Yes
1  France  32.5  40000.0        No
2   Italy  41.0  37000.0       Yes
3  France  35.0  40000.0        No
4   India  30.0  51000.0       Yes

One Hot Encoding:
   France  India  Italy
0   False   True  False
1    True  False  False
2   False  False   True
3    True  False  False
```



```
Updated Dataset:
   France  India  Italy   Age   Salary Purchased
0   False   True  False  29.0  32000.0       Yes
1    True  False  False  32.5  40000.0        No
2   False  False   True  41.0  37000.0       Yes
3    True  False  False  35.0  40000.0        No
4   False   True  False  30.0  51000.0       Yes


DataFrame Information After Preprocessing:
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    5 non-null      str
 1   Age        5 non-null      float64
 2   Salary     5 non-null      float64
 3   Purchased  5 non-null      str
dtypes: float64(2), str(2)
memory usage: 192.0 bytes
None


Final Dataset:
   France  India  Italy   Age   Salary Purchased
0   False   True  False  29.0  32000.0         1
1    True  False  False  32.5  40000.0         0
2   False  False   True  41.0  37000.0         1
3    True  False  False  35.0  40000.0         0
4   False   True  False  30.0  51000.0         1
```

Result:

Hence, the data pre-processing in data science is analysed.

Exp:4. Handling Missing and Inappropriate Data in a Dataset

Code:

```
import numpy as np
import pandas as pd

# Creating the original dataset
df = pd.DataFrame({
    'CustomerID': [1, 2, 3, 4, 5],
    'Age-group': ['20-25', '30-35', '25-30', '20-25', '35+'],
    'Rating (1-5)': [4, 5, 6, -1, 3],
    'Hotel': ['Ibis', 'Lemon Tree', 'RedFox', 'Lemon Tree', 'Ibis'],
    'FoodPreference': ['veg', 'Non-veg', 'veg', 'veg', 'Vegetarian'],
    'Bill': [1300, 2000, 1322, 1234, 989],
    'NoOfPax': [2, 3, 2, 2, 2],
    'Estimated Salary': [40000, 59000, 30000, 1200000, 45000],
    'Age_group.1': ['20-25', '30-35', '25-30', '20-25', '35+']
})

print("ORIGINAL DATASET")
print(df)

print("\nDUPLICATE ROWS")
# (Assuming df.duplicated() goes here based on context)
```

```
df.reset_index(drop=True, inplace=True)
df.drop(['Age_group.1'], axis=1, inplace=True)

df.loc[(df['Rating (1-5)'] < 1) | (df['Rating (1-5)'] > 5), 'Rating (1-5)'] =
np.nan

df.loc[df['Bill'] < 0, 'Bill'] = np.nan
df.loc[df['Estimated Salary'] < 0, 'Estimated Salary'] = np.nan
df.loc[(df['NoOfPax'] < 1) | (df['NoOfPax'] > 20), 'NoOfPax'] = np.nan

print("\nUNIQUE AGE GROUPS")
print(df['Age-group'].unique())
print("\nUNIQUE HOTELS")
print(df['Hotel'].unique())
print("\nUNIQUE FOOD PREFERENCES")
print(df['FoodPreference'].unique())

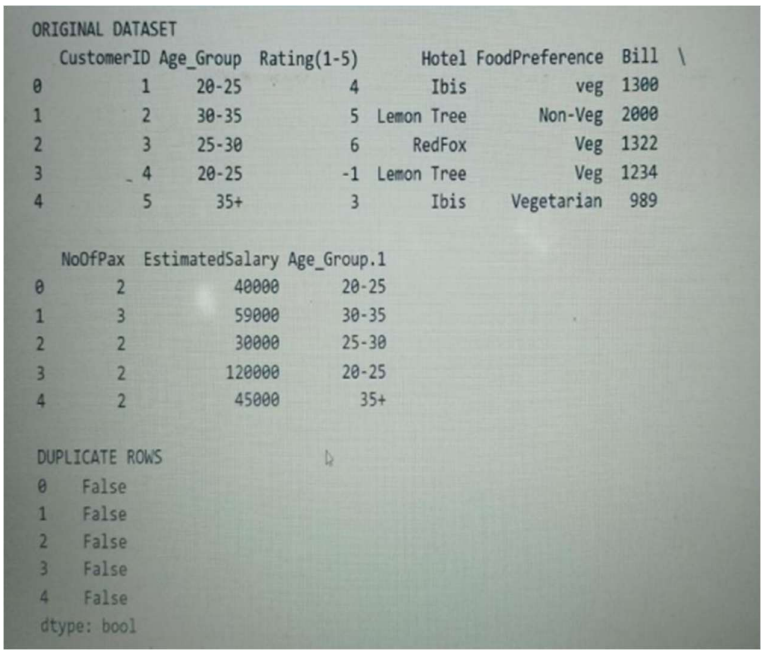
df['FoodPreference'] = df['FoodPreference'].replace(['non-veg', 'non -
veg'], 'Non-veg')

df['Estimated Salary'] = df['Estimated
Salary'].fillna(round(df['Estimated Salary'].mean()))
df['NoOfPax'] = df['NoOfPax'].fillna(round(df['NoOfPax'].median()))
df['Bill'] = df['Bill'].fillna(round(df['Bill'].mean()))
```

print("\nFINAL CLEANED DATASET")

print(df)

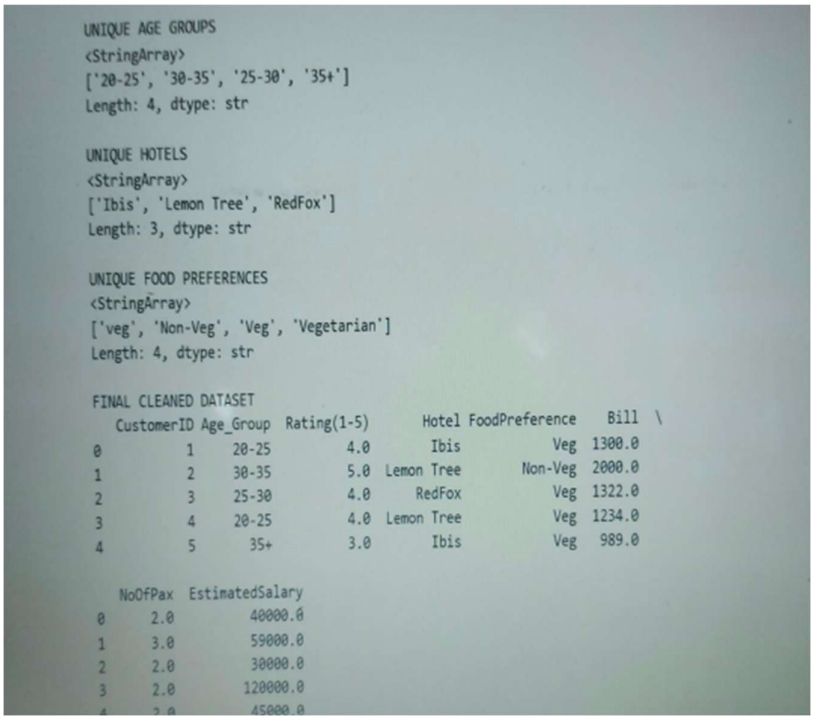
Output:



```
ORIGINAL DATASET
   CustomerID Age_Group  Rating(1-5)       Hotel FoodPreference  Bill  \
0           1     20-25           4        Ibis            veg  1300
1           2     30-35           5  Lemon Tree        Non-Veg  2000
2           3     25-30           6     RedFox            Veg  1322
3           4     20-25          -1  Lemon Tree            Veg  1234
4           5       35+           3        Ibis     Vegetarian   989

   NoOfPax  EstimatedSalary Age_Group.1
0        2            40000       20-25
1        3            59000       30-35
2        2            30000       25-30
3        2           120000       20-25
4        2            45000         35+

DUPLICATE ROWS
0    False
1    False
2    False
3    False
4    False
dtype: bool
```



```
UNIQUE AGE GROUPS
<StringArray>
['20-25', '30-35', '25-30', '35+']
Length: 4, dtype: str

UNIQUE HOTELS
<StringArray>
['Ibis', 'Lemon Tree', 'RedFox']
Length: 3, dtype: str

UNIQUE FOOD PREFERENCES
<StringArray>
['veg', 'Non-Veg', 'Veg', 'Vegetarian']
Length: 4, dtype: str

FINAL CLEANED DATASET
   CustomerID Age_Group  Rating(1-5)       Hotel FoodPreference    Bill  \
0           1     20-25          4.0        Ibis            Veg  1300.0
1           2     30-35          5.0  Lemon Tree        Non-Veg  2000.0
2           3     25-30          4.0     RedFox            Veg  1322.0
3           4     20-25          4.0  Lemon Tree            Veg  1234.0
4           5       35+          3.0        Ibis            Veg   989.0

   NoOfPax  EstimatedSalary
0      2.0          40000.0
1      3.0          59000.0
2      2.0          30000.0
3      2.0         120000.0
4      2.0          45000.0
```

Result

Handling Missing and Inappropriate Data in a Dataset analysed.

Exp No:5 Experiment to detect outliers in a given data set.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

array = np.array([10, 15, 19, 22, 25, 28, 31, 34, 37, 39, 42, 44, 47, 49,
110, 140])

print("Array:")
print(array)

print("Mean:")
print(array.mean())

print("25th Percentile:")
print(np.percentile(array, 25))

print("50th Percentile:")
print(np.percentile(array, 50))

print("75th Percentile:")
print(np.percentile(array, 75))

print("100th percentile:")
```

```
print(np.percentile(array, 100))

def outDetection(array):
    Q1, Q3 = np.percentile(array, [25, 75])
    IQR = Q3 - Q1
    lr = Q1 - (1.5 * IQR)
    ur = Q3 + (1.5 * IQR)
    return lr, ur

lr, ur = outDetection(array)
print("Lower Range:")
print(lr)
print("Upper Range:")
print(ur)

plt.figure(figsize=(5, 2.5))
plt.hist(array, bins=5)
plt.show()

new_array = array[(array > lr) & (array < ur)]
print("Array after removing outliers:")
print(new_array)

plt.figure(figsize=(5, 2.5))
```

```
plt.hist(new_array, bins=5)
plt.show()

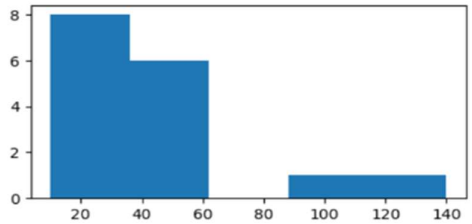
Ir1, ur1 = outDetection(new_array)
print("New Lower Range:")
print(Ir1)
print("New Upper Range:")
print(ur1)

final_array = new_array[(new_array > Ir1) & (new_array < ur1)]
print("Final Array:")
print(final_array)

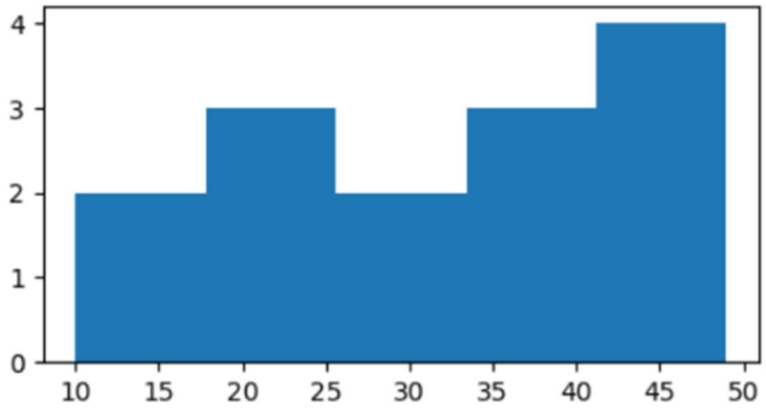
plt.figure(figsize=(5, 2.5))
plt.hist(final_array, bins=5)
plt.show()
```

Output:

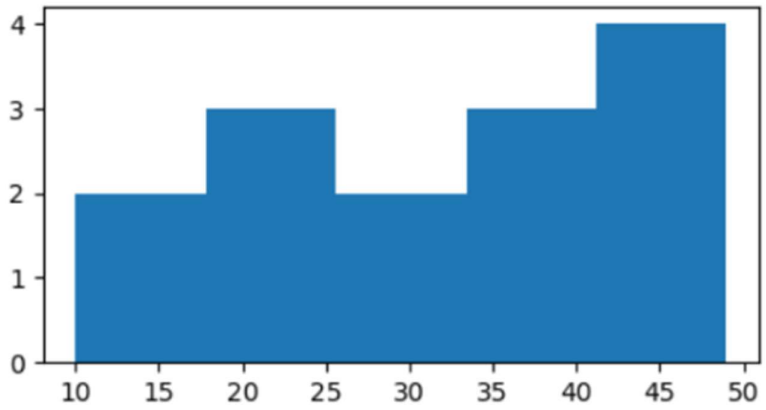
```
Array:
[ 10  15  19  22  25  28  31  34  37  39  42  44  47  49 110 140]
Mean:
43.25
25th Percentile:
24.25
50th Percentile:
35.5
75th Percentile:
44.75
100th Percentile:
140.0
Lower Range:
-6.5
Upper Range:
75.5
```



```
Array After Removing Outliers:
[10 15 19 22 25 28 31 34 37 39 42 44 47 49]
```




```
New Lower Range:
-5.0
New Upper Range:
69.0
Final Array:
[10 15 19 22 25 28 31 34 37 39 42 44 47 49]
```



Result:

Hence, the procedure to identify the outliers in a given dataset is analysed.

Exp No:6. Experiment to understand feature scaling

Code:

```
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler

df = pd.DataFrame({
   'Country': ['USA', 'France', 'USA', 'China', 'France', 'USA', 'China', 'France', 'USA', 'China'],
   'Age': [22, 34, 27, np.nan, 45, 38, 31, 50, 29, 41],
   'Salary': [35000, 60000, 48000, 72000, np.nan, 55000, 68000, 80000, 52000, 64000],
   'purchased': ['NO', 'Yes', 'NO', 'Yes', 'NO', 'Yes', 'NO', 'Yes', 'NO', 'Yes']
})

print("Original Dataset.")
print(df)

print("\n\nDataFrame Information")
print(df.info())
```

```
df['Country'] = df['Country'].fillna(df['Country'].mode()[0])

features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values

age = SimpleImputer(strategy="mean", missing_values=np.nan)
Salary = SimpleImputer(strategy="mean", missing_values=np.nan)

age.fit(features[:, [1]])
Salary.fit(features[:, [2]])

features[:, [1]] = age.transform(features[:, [1]])
features[:, [2]] = Salary.transform(features[:, [2]])

print("\n\nAfter Filling Missing Values.")
print(features)

oh = OneHotEncoder(sparse_output=False)
country = oh.fit_transform(features[:, [0]])

print("\n\nOneHot Encoding:")
print(country)

final_set = np.concatenate((country, features[:, [1, 2]]), axis=1)
```

```
print("\n\nFinal set:")
print(final_set)

sc = StandardScaler()
sc.fit(final_set)
feat_standard_scaler = sc.transform(final_set)

print("\n\nStandard Scaler:")
print(feat_standard_scaler)

mms = MinMaxScaler(feature_range=(0, 1))
mms.fit(final_set)
feat_minmax_scaler = mms.transform(final_set)

print("\n\nMin Max Scaler:")
print(feat_minmax_scaler)
```

Output:

```
Original Dataset:
   Country    Age    Salary Purchased
0      USA  22.0  35000.0         No
1   France  34.0  60000.0        Yes
2      USA  27.0  48000.0         No
3    China   NaN  72000.0        Yes
4   France  45.0      NaN         No
5      USA  38.0  55000.0        Yes
6    China  31.0  68000.0         No
7      NaN  50.0  80000.0        Yes
8   France  29.0  52000.0         No
9      USA  41.0  64000.0        Yes


DataFrame Information:
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #    Column      Non-Null Count  Dtype
---   ------      --------------  -----
 0    Country     9 non-null      str
 1    Age         9 non-null      float64
 2    Salary      9 non-null      float64
 3    Purchased   10 non-null     str
dtypes: float64(2), str(2)
memory usage: 312.0 bytes
None
After Filling Missing Values:
[['USA' 22.0 35000.0]
 ['France' 34.0 60000.0]
 ['USA' 27.0 48000.0]
 ['China' 35.22222222222222 72000.0]
 ['France' 45.0 59333.333333333336]
 ['USA' 38.0 55000.0]
 ['China' 31.0 68000.0]
 ['USA' 50.0 80000.0]
 ['France' 29.0 52000.0]
 ['USA' 41.0 64000.0]]


One Hot Encoding:
[[0. 0. 1.]
 [0. 1. 0.]
 [0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]
 [1. 0. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

```
Final Set:
[[0.0 0.0 1.0 22.0 35000.0]
 [0.0 1.0 0.0 34.0 60000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [1.0 0.0 0.0 35.22222222222222 72000.0]
 [0.0 1.0 0.0 45.0 59333.333333333336]
 [0.0 0.0 1.0 38.0 55000.0]
 [1.0 0.0 0.0 31.0 68000.0]
 [0.0 0.0 1.0 50.0 80000.0]
 [0.0 1.0 0.0 29.0 52000.0]
 [0.0 0.0 1.0 41.0 64000.0]]


Standard Scaler:
[[-5.00000000e-01 -6.54653671e-01  1.00000000e+00 -1.63305082e+00
  -2.00154053e+00]
 [-5.00000000e-01  1.52752523e+00 -1.00000000e+00 -1.50954278e-01
   5.48367268e-02]
 [-5.00000000e-01 -6.54653671e-01  1.00000000e+00 -1.01551060e+00
  -9.32224356e-01]
 [ 2.00000000e+00 -6.54653671e-01 -1.00000000e+00  0.00000000e+00
   1.04189781e+00]
 [-5.00000000e-01  1.52752523e+00 -1.00000000e+00  1.20763422e+00
   5.98484550e-16]
 [-5.00000000e-01 -6.54653671e-01  1.00000000e+00  3.43077904e-01
  -3.56438724e-01]
 [ 2.00000000e+00 -6.54653671e-01 -1.00000000e+00 -5.21478414e-01
   7.12877448e-01]
 [-5.00000000e-01 -6.54653671e-01  1.00000000e+00  1.82517445e+00
   1.69993853e+00]
 [-5.00000000e-01  1.52752523e+00 -1.00000000e+00 -7.68494505e-01
  -6.03203995e-01]
 [-5.00000000e-01 -6.54653671e-01  1.00000000e+00  7.13602040e-01
   3.83857088e-01]]
```

Result:

Hence, the importance of feature scaling is understood.

Exp No:7. Experiment to understand EDA-Quantitative and Qualitative analysis.

Code:

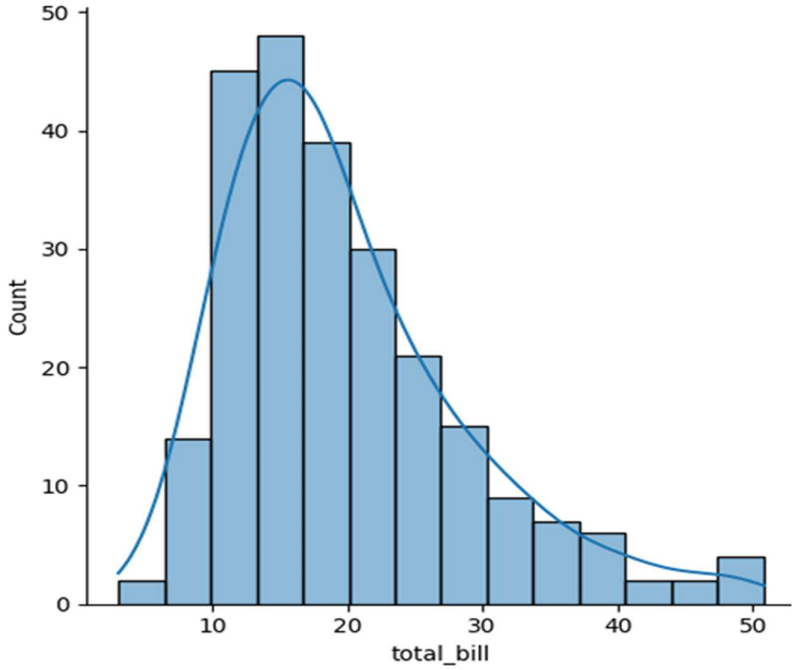
```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
tips = sns.load_dataset('tips')
tips.head()
sns.displot(tips.total_bill, kde=True)
sns.displot(tips.total_bill, kde=False)
sns.jointplot(x=tips.tip, y=tips.total_bill)
sns.jointplot(x=tips.tip, y=tips.total_bill, kind="reg")
sns.jointplot(x=tips.total_bill, kind="hex")
sns.pairplot(tips)
tips.time.value_counts()
sns.pairplot(tips, hue='time')
sns.pairplot(tips, hue='day')
sns.heatmap(tips.corr(numeric_only=True), annot=True)
sns.boxplot(tips.total_bill)
sns.boxplot(tips.tip)
sns.countplot(tips.day)
sns.countplot(tips.sex)
```

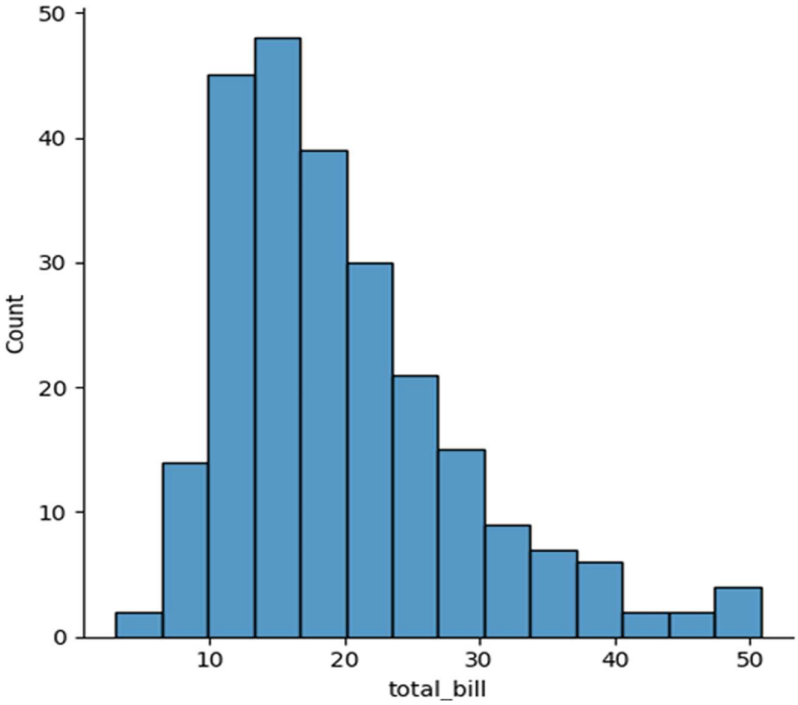
tips.sex.value_counts().plot(kind='pie')

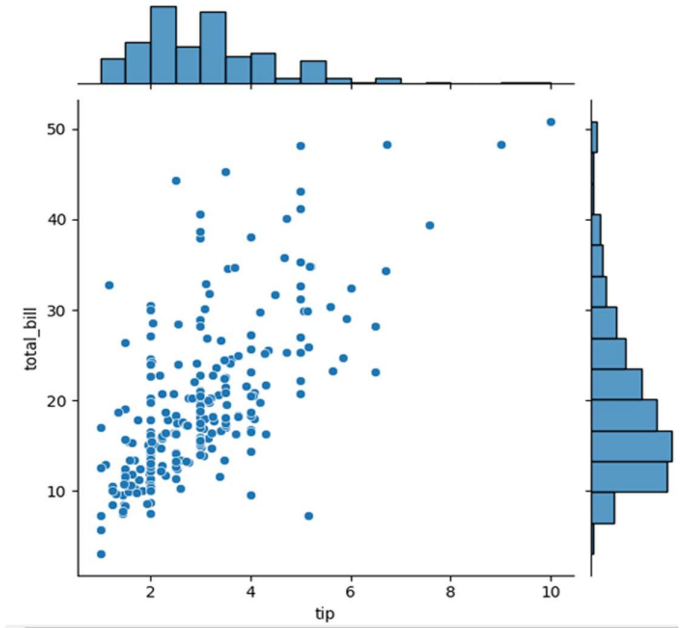
tips.sex.value_counts().plot(kind='bar')

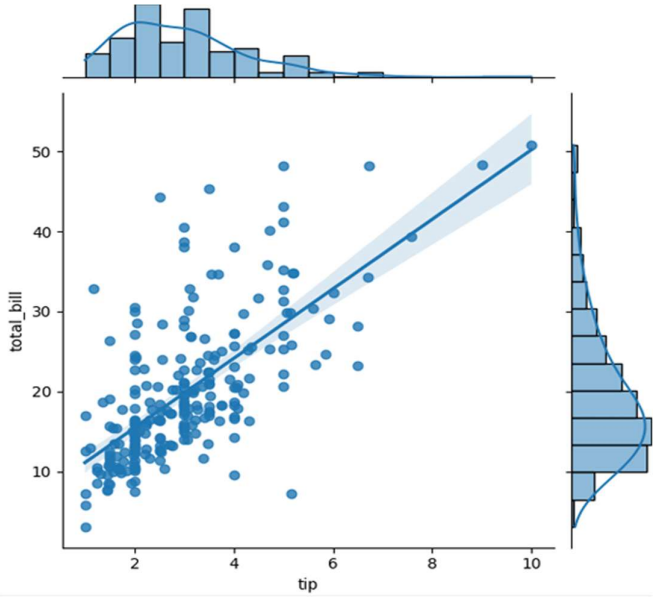
sns.countplot(tips[tips.time == 'Dinner']['day'])

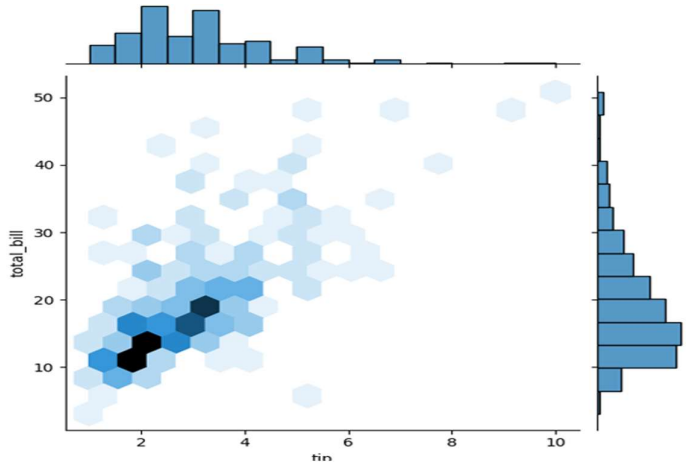
Output:

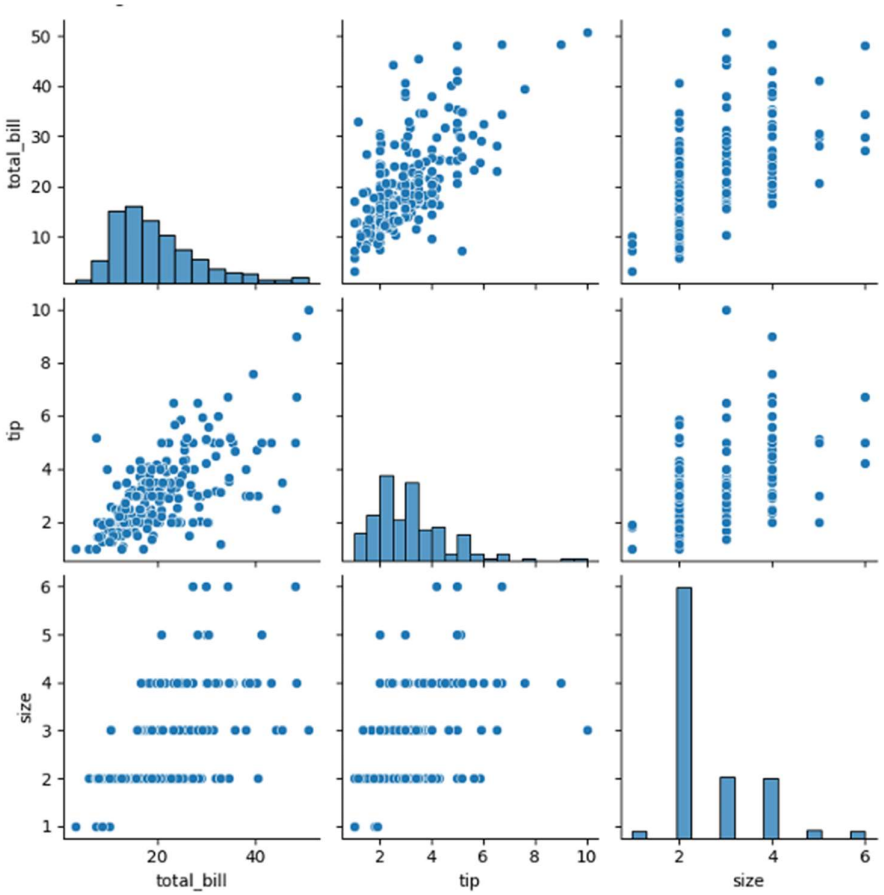


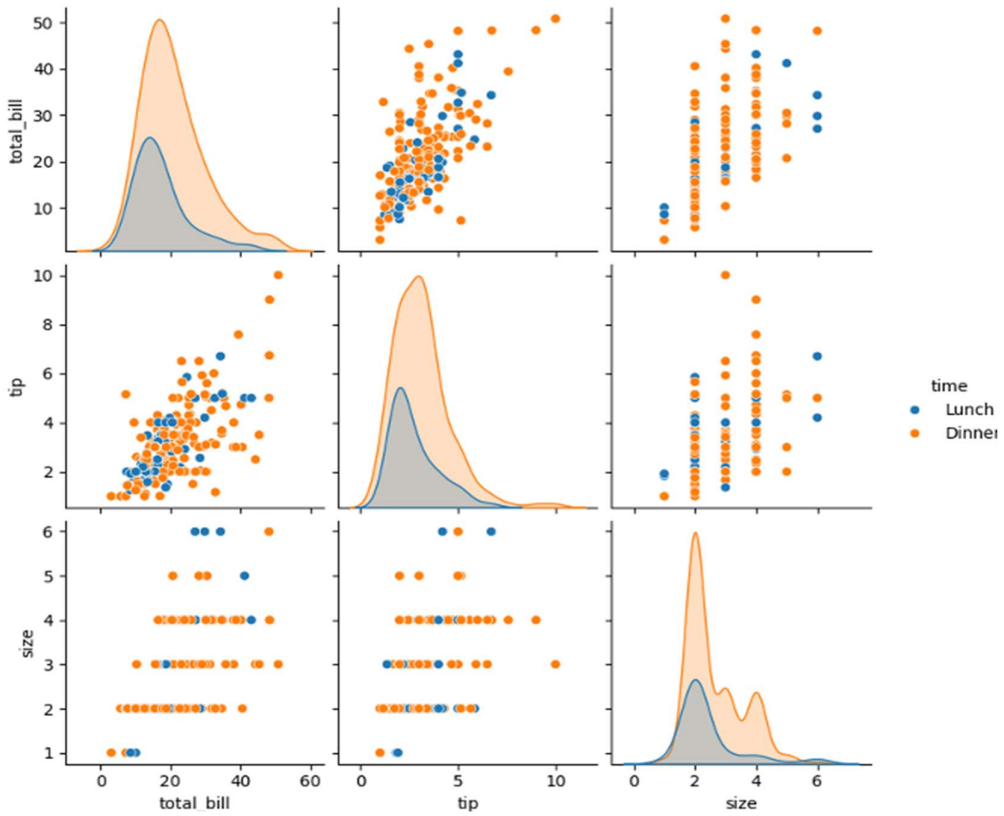



total_bill
tip
10
20
30
40
50
2
4
6
8
10


total_bill
tip
10
20
30
40
50
2
4
6
8
10


total_bill
tip
10
20
30
40
50
2
4
6
8
10


total_bill
tip
size
total_bill
tip
size


total_bill
tip
size
total_bill
tip
size
time
Lunch
Dinner

Exp No:8. Experiment to understand Linear Regression for a given data set.

Code:

```
import numpy as np

import pandas as pd


df = pd.read_csv('Salary-data.csv')

df.info()

df.dropna(inplace=True)

df.info()

df.describe()


features = df.iloc[:, [0]].values

label = df.iloc[:, [1]].values


from sklearn.model_selection import train_test_split


x_train, x_test, y_train, y_test = train_test_split(

    features, label, test_size=0.2, random_state=1

)


from sklearn.linear_model import LinearRegression


model = LinearRegression()
```

```
model.fit(x_train, y_train)

model.score(x_train, y_train)
model.score(x_test, y_test)

model.coef_
model.intercept_

import pickle

pickle.dump(model, open('salarypred.model', 'wb'))

model = pickle.load(open('salarypred.model', 'rb'))

yr_of_exp = float(input("enter year"))
yr_of_exp_NP = np.array([[yr_of_exp]])
Salary = model.predict(yr_of_exp_NP)

print(
   "Estimated salary for {} year of experience is {}".format(
      yr_of_exp, Salary
   )
)
```

Output:

	YearsExperience	Salary
count	30.000000	30.000000
mean	5.313333	76003.000000
std	2.837888	27414.429785
min	1.100000	37731.000000
25%	3.200000	56720.750000
50%	4.700000	65237.000000
75%	7.700000	100544.750000
max	10.500000	122391.000000

```
▾ LinearRegression
LinearRegression()
```

```
0.9603182547438908
```

```
array([[9281.30847068]])
```

```
array([27166.73682891])
```

Result:

Hence , This code has executed successfully . linear regression is verified.

Exp No:9. Experiment to understand Logistic Regression for a given data set.

Code:

```
import numpy as np

import pandas as pd


df = pd.read_csv('Social_Network_Ads.csv')


data = {

   'UserID': range(1, 21),

   'Gender': [

      'Female',

      'Male',

      'Female',

      'Male',

      'Male',

      'Female',

      'Male',

      'Female',

      'Male',

      'Female',

      'Male',

      'Female',

      'Male',
```

```
    'Female',
    'Male',
    'Female',
    'Male',
    'Female',
    'Male',
    'Female',
],
'Age': [
    21,
    29,
    34,
    40,
    27,
    32,
    46,
    25,
    37,
    43,
    30,
    48,
    23,
    35,
    41,
```

```
    28,
    34,
    26,
    45,
    33,
],
'EstimatedSalary': [
    28000,
    45000,
    52000,
    88000,
    36000,
    58000,
    105000,
    30000,
    75000,
    92000,
    52000,
    115000,
    27000,
    90000,
    98000,
    41000,
    82000,
```

```
        64000,
        108000,
        60000,
    ],
    'Purchased': [0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1],
}

df = pd.DataFrame(data)
print(df)
print(df.head())

features = df.iloc[:, [2, 3]].values
label = df.iloc[:, [4]].values
print(label)

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

for i in range(1, 101):
    x_train, x_test, y_train, y_test = train_test_split(
        features, label, test_size=0.2, random_state=i
    )

    model = LogisticRegression()
```

```
    model.fit(x_train, y_train)
    train_score = model.score(x_train, y_train)
    test_score = model.score(x_test, y_test)

    if test_score > train_score:
        print(
            "Test:",
            test_score,
            "Train:",
            train_score,
            "Random state:",
            i,
        )

x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=42
)

finalModel = LogisticRegression()
finalModel.fit(x_train, y_train)

print(
    "Training score:",
    finalModel.score(x_train, y_train),
```

)

print("Testing score:", finalModel.score(x_test, y_test))

from sklearn.metrics import classification_report

print(classification_report(label, finalModel.predict(features)))

Output:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

```
array([[    19,  19000],
       [    35,  20000],
       [    26,  43000],
       [    27,  57000],
       [    19,  76000],
       [    27,  58000],
       [    27,  84000],
       [    32, 150000],
       [    25,  33000],
       [    35,  65000],
       [    26,  80000],
       [    26,  52000],
       [    20,  86000],
       [    32,  18000],
       [    18,  82000],
       [    29,  80000],
       [    47,  25000],
       [    45,  26000],
       [    46,  28000]
```

```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0,
       1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0,
       1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
       0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1,
       1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1,
       0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0,
       1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1,
       0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1,
       1, 1, 0, 1], dtype=int64)
```

Result:

Hence , this code has successfully executed and output is been verified.

Exp No:10. Experiment to understand K-means clustering algorithm for a given data set.

Code:

```
import pandas as pd

df = pd.read_csv(list(uploaded.keys())[0])

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.pairplot(df)
plt.show()

features = df.iloc[:, [3, 4]].values

from sklearn.cluster import KMeans

model = KMeans(n_clusters=5, random_state=42)
model.fit(features)

Final = df.iloc[:, [3, 4]].copy()
Final['label'] = model.predict(features)
```

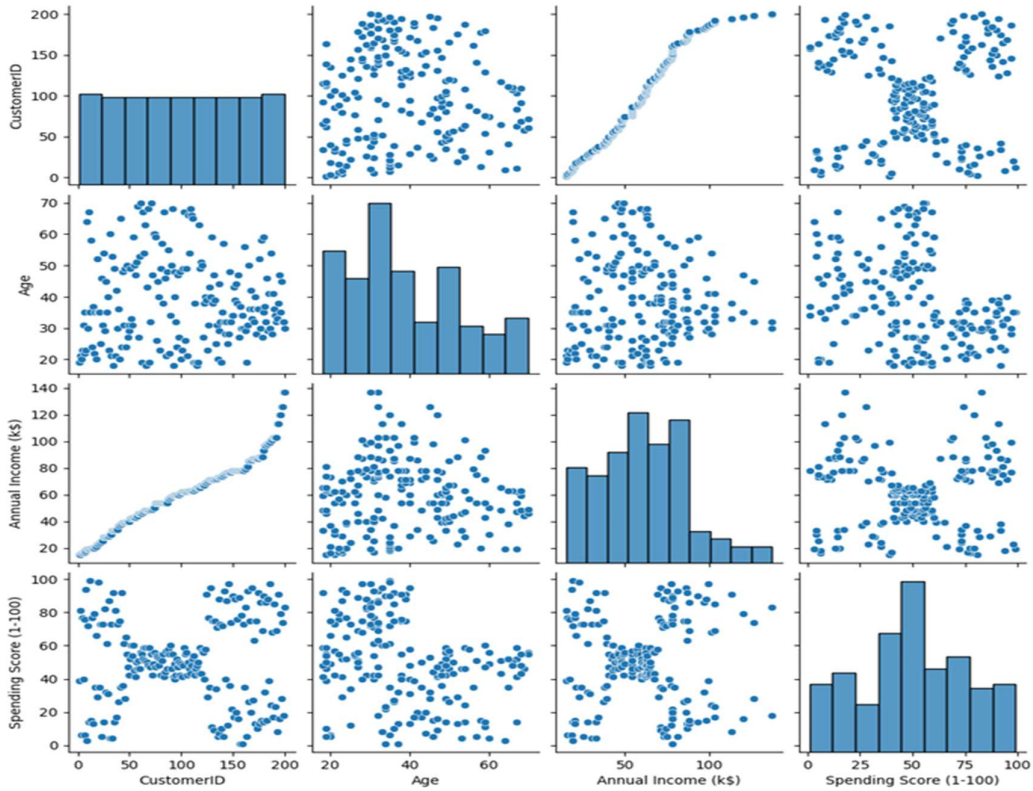
```
Final.head()

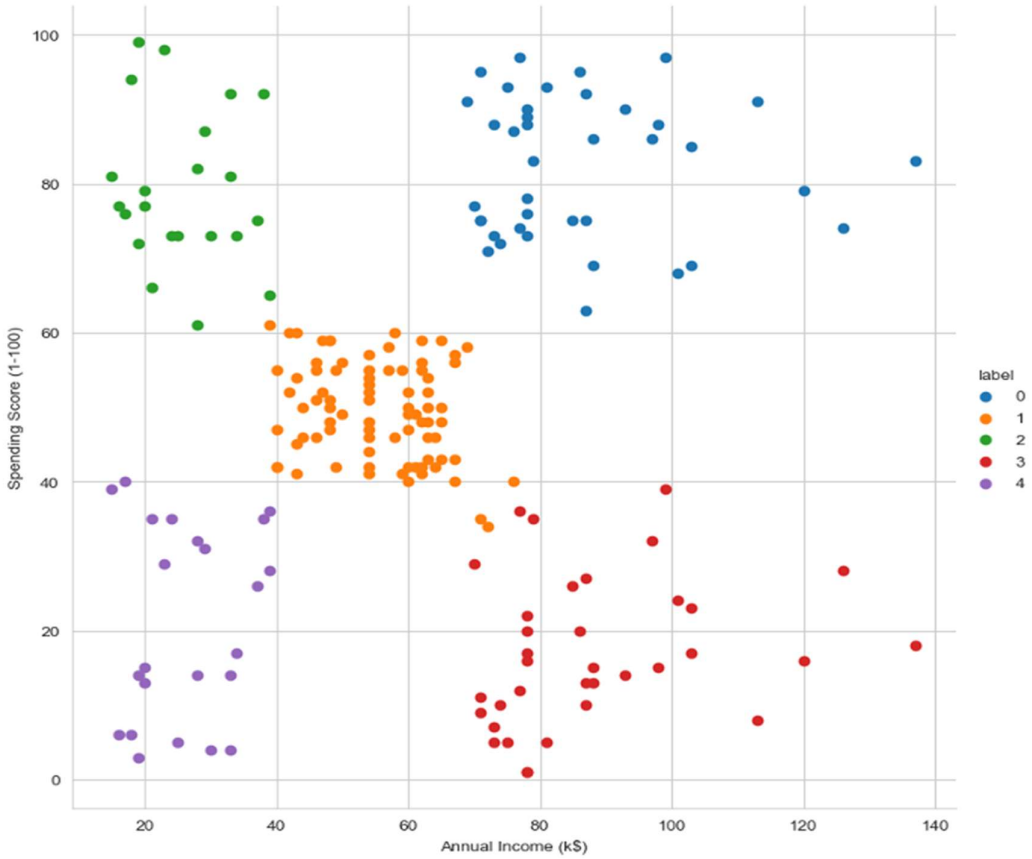

sns.set_style("whitegrid")
sns.FacetGrid(Final, hue="label", height=8).map(
    plt.scatter, "Annual Income(k$)", "Spending score (1-100)"
).add_legend()

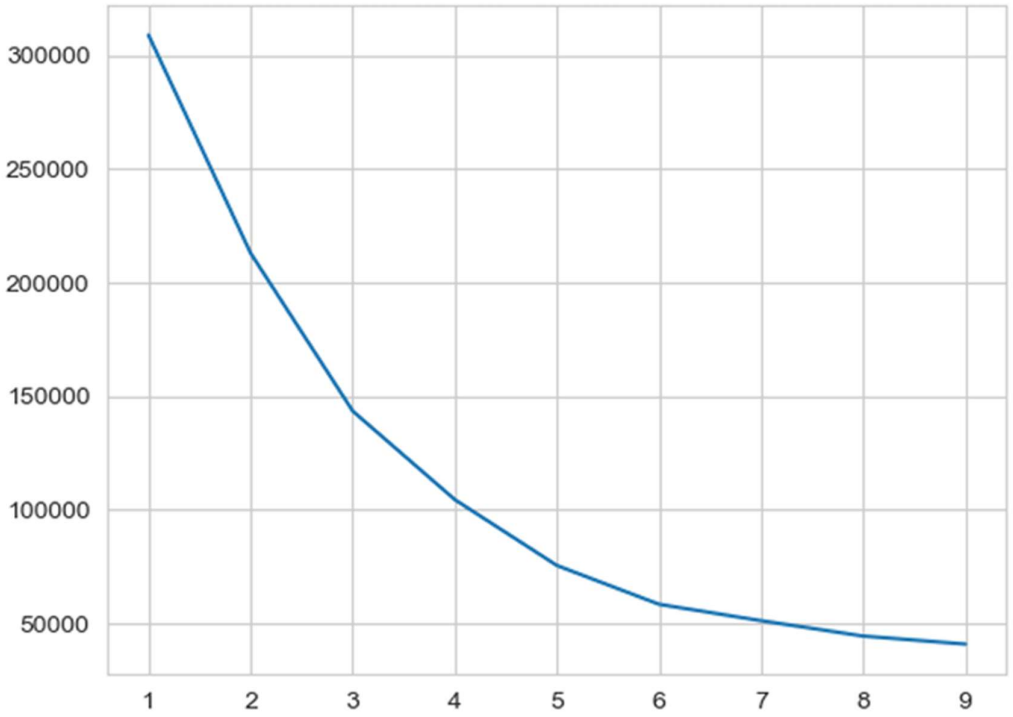
plt.show()
```

Output:

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40


CustomerID
Age
Annual Income (k$)
Spending Score (1-100)


Spending Score (1-100)
Annual Income (k$)
label
0
1
2
3
4



Result:

Thus the code has successfully executed and output is been analysed and it is been verified.

Exp No:11. Experiment to understand KNN algorithm for a given data set

Code:

```
import pandas as pd

data = pd.read_csv(list(uploaded.keys())[0])
data.head()

import numpy as np
import pandas as pd

data.info()
print(data['variety'].value_counts())
print(data.head())

X = data.iloc[:, 1:-1].values
y = data.iloc[:, -1].values

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
```

```
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(x_train, y_train)

print(knn.score(x_train, y_train))
print(knn.score(x_test, y_test))

from sklearn.metrics import classification_report, confusion_matrix

prediction = knn.predict(x_test)
print(confusion_matrix(y_test, prediction))
print(classification_report(y_test, prediction))
```

Output:

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa

```
array([[50,  0,  0],
       [ 0, 47,  3],
       [ 0,  2, 48]], dtype=int64)
```

Result:

Thus the experiment of knn was executed successfully and the output was verified.

Experiment 12: Conduct Z test for the Data Given using Python Objective

Code:

```
import numpy as np
from math import sqrt
from scipy import stats
from scipy.stats import norm

print("=== EXP NO: 12: Z-TEST, t-TEST, F-TEST ANOVA ===\n")

# 1. One-Sample t-Test[span_2](start_span)[span_2](end_span)
print("--- 1. One-Sample t-Test ---")
marks = np.array([82, 79, 85, 88, 76, 81, 84, 80, 86, 83])
mu_0_t = 86
t_stat, p_value_t = stats.ttest_1samp(marks, mu_0_t)

print(f"T-Statistic: {t_stat:.3f}")
print(f"P-value: {p_value_t:.4f}")

alpha = 0.05
if p_value_t < alpha:
    print("Result: Reject Null Hypothesis\n")
else:
    print("Result: Fail to Reject Null Hypothesis\n")
```

```
# 2. Z-Test[span_3](start_span)[span_3](end_span)
print("--- 2. Z-Test ---")
x_bar = 49.5
mu_0_z = 50
sigma = 4
n = 64

z_stat = (x_bar - mu_0_z) / (sigma / sqrt(n))
p_value_z = 2 * (1 - norm.cdf(abs(z_stat)))

print(f"Z-statistics: {z_stat:.3f}")
print(f"P-value: {p_value_z:.4f}")

if p_value_z < alpha:
    print("Result: Reject Null Hypothesis\n")
else:
    print("Result: Fail to Reject Null Hypothesis\n")



# 3. One-Way ANOVA (F-
Test)[span_4](start_span)[span_4](end_span)
print("--- 3. One-Way ANOVA (F-Test) ---")
```

```
A = [15, 18, 17]

B = [22, 24, 23]

C = [19, 21, 20]


f_stat, p_value_f = stats.f_oneway(A, B, C)


print(f"F-statistics: {f_stat:.3f}")

print(f"P-value: {p_value_f:.4f}")


if p_value_f < alpha:

   print("Result: Reject Null Hypothesis")

else:

   print("Result: Fail to Reject Null Hypothesis")
```

Output:

```
Sample Mean: 150.20

Z-Statistic: 0.6406

P-Value: 0.5218

Fail to reject the null hypothesis: There is no significant difference in average weight from 150 grams.
```

Result:

Thus the experiment was executed successfully and Z test for the Data Given using Python Objective was analysed.

Exp No:13.a. Conduct an experiment to show data visualization using line plot

Code:

```
import matplotlib.pyplot as cricket

 Overs=list(range(5,51,5))
Indian_Score=[30,55,90,129,165,200,239,270,310,350]
Srilankan_Score=[25,70,90,120,140,170,195,220,255,279]
cricket.plot(Overs,Indian_Score)

cricket.plot(Overs,Srilankan_Score)

cricket.show()

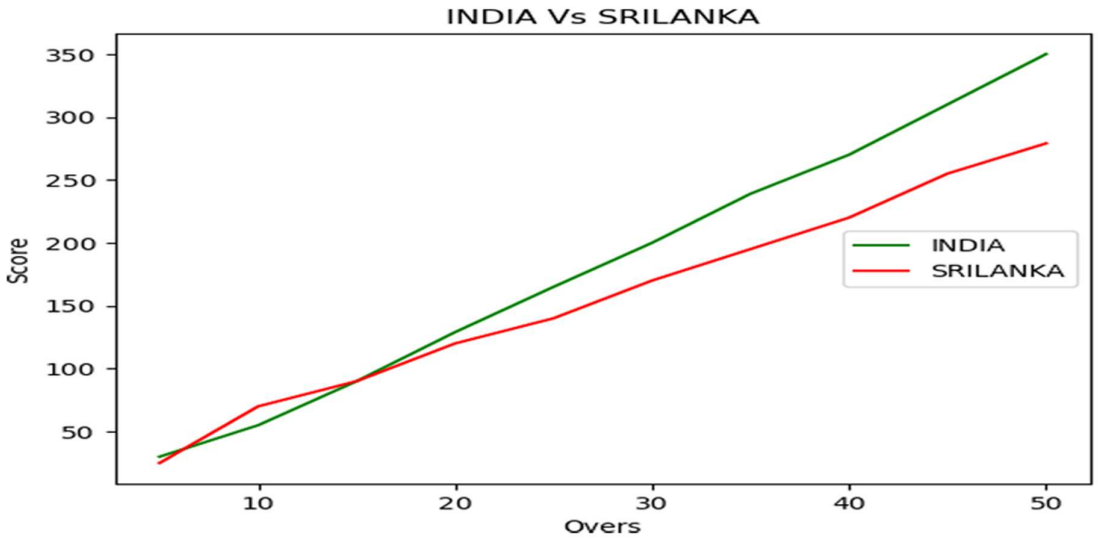
cricket.title("INDIA Vs SRILANKA")

cricket.xlabel("Overs")

cricket.ylabel("Score")

cricket.legend()
cricket.plot(Overs,Indian_Score,color="green",label="INDIA")
cricket.plot(Overs,Srilankan_Score,color="red",label="SRILANKA")
cricket.legend(loc="center right")
```

Output:



Result:

Hence, the experiment to show data visualization using line plot

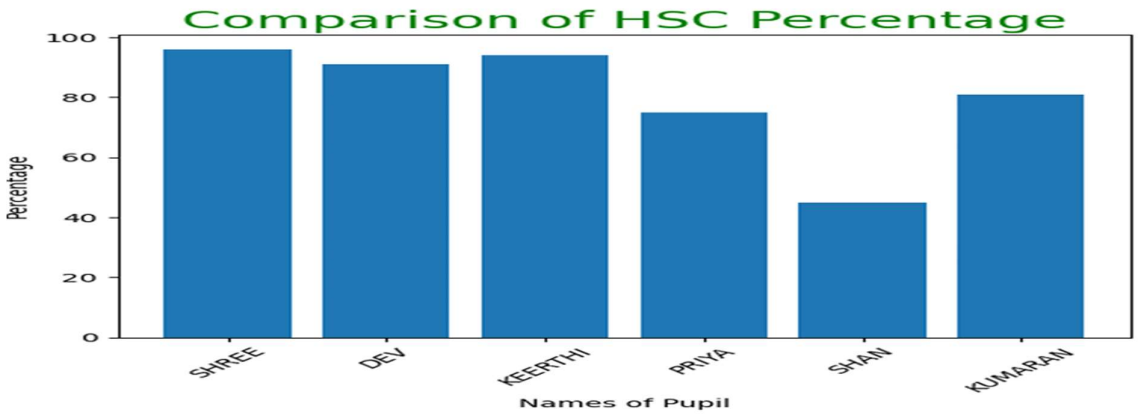
Was analysed and executed successfully.

Exp No:13.b Conduct an experiment to show data visualization using bar chart

Code:

```
import matplotlib.pyplot as hscmark
import numpy as np
 Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']
xaxis = np.arange(len(Names))
Percentage_hsc = [96, 91, 94, 75, 45, 81]
hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")
 hscmark.show().
```

Output:



Result:

Hence , experiment to show data visualization using bar chart

Was analysed and executed successfully.

Exp No:13.c Conduct an experiment to show data visualization using pie chart.

Code:

```
import matplotlib.pyplot as election

labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']

Votes = [315, 130, 245, 210]

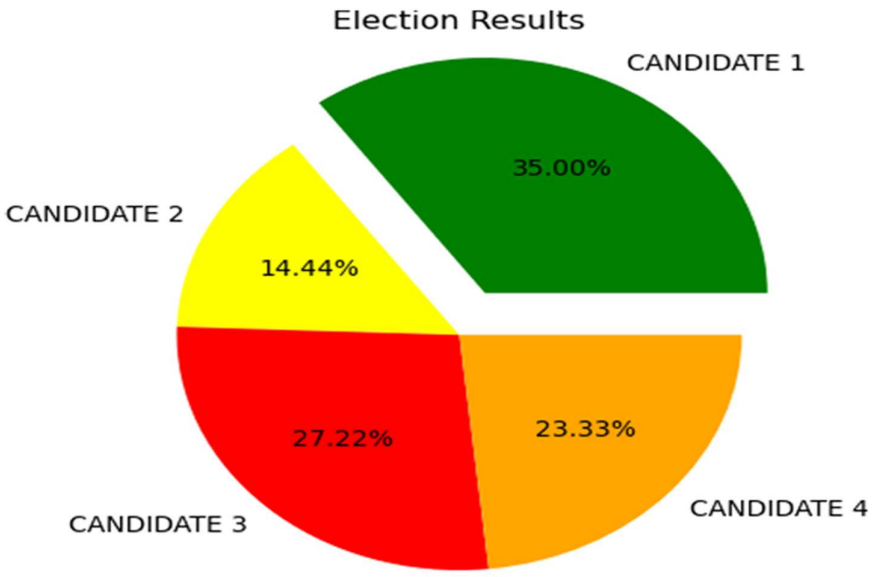
colors = ['green', 'yellow', 'red', 'orange']

explode = (0.2, 0, 0, 0) election.pie(Votes, lab

els=(labels, colors=colors, explode=explode, autopct='%0.2f%%') election.title('Election Results')

 election.show()
```

Output:



Result:

Hence , the experiment to show data visualization using pie chart was analysed and executed successfully.

Exp No:14. To Count the frequency of occurrence of a word in a body of text is often needed during text processing.

Code:

```
import nltk from nltk.tokenize

 import word_tokenize from nltk.corpus

import gutenberg

nltk.download('punkt')

sample = gutenberg.raw("austen-emma.txt")

token = word_tokenize(sample)

wlist = []

for i in range(50):

wlist.append(token[i])

wordfreq = [wlist.count(w) for w in wlist]

print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

Output:

```
Pairs
[('[', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816',
1), (']', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1), ('I', 2), ('Emma
', 2), ('Woodhouse', 1), (',', 5), ('handsome', 1), (',', 5), ('clever'
, 1), (',', 5), ('and', 3), ('rich', 1), (',', 5), ('with', 2), ('a', 1
), ('comfortable', 1), ('home', 1), ('and', 3), ('happy', 1), ('disposi
tion', 1), (',', 5), ('seemed', 1), ('to', 1), ('unite', 1), ('some', 1
), ('of', 2), ('the', 2), ('best', 1), ('blessings', 1), ('of', 2), ('e
xistence', 1), (';', 1), ('and', 3), ('had', 1), ('lived', 1), ('nearly
', 1), ('twenty-one', 1), ('years', 1), ('in', 1), ('the', 2), ('world'
, 1), ('with', 2)]
```

Result:

Hence , Count the frequency of occurrence of a word in a body of text is often needed during text processing was executedsuccessfully.

Exp No:15. Data Collection and Initial Exploration

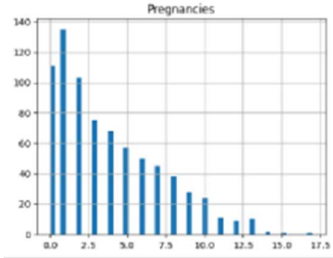
Code:

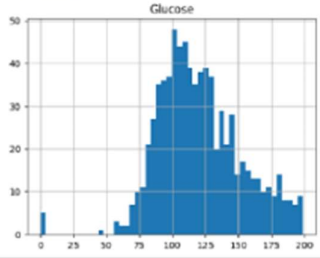
```
import pandas as pd
db = pd.read_csv('/content/drive/MyDrive/PIMA Indian dataset.csv')
print(db.head(1))
print(db.info())
print(db.describe())
import matplotlib.pyplot as plt
import seaborn as sns
db.hist(bins=50, figsize=(20, 15))
plt.show()
sns.pairplot(db)
plt.show()
```

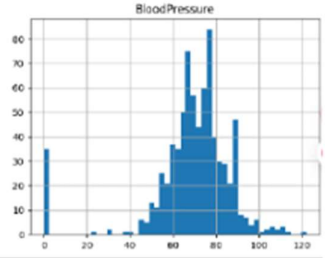
Output:

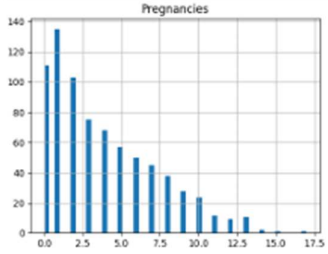
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      3.845052  120.894531      69.105469      20.536458   79.799479
std       3.369578   31.972618      19.355807      15.952218  115.244002
min       0.000000    0.000000       0.000000       0.000000    0.000000
25%       1.000000   99.000000      62.000000       0.000000    0.000000
50%       3.000000  117.000000      72.000000      23.000000   30.500000
75%       6.000000  140.250000      80.000000      32.000000  127.250000
max      17.000000  199.000000     122.000000      99.000000  846.000000
```

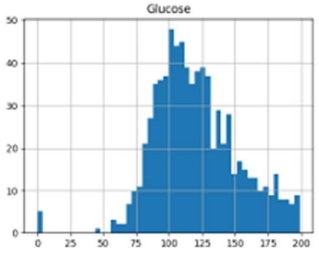
	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000
mean	31.992578	0.471876	33.240885	0.348958
std	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.078000	21.000000	0.000000
25%	27.300000	0.243750	24.000000	0.000000
50%	32.000000	0.372500	29.000000	0.000000
75%	36.600000	0.626250	41.000000	1.000000
max	67.100000	2.420000	81.000000	1.000000

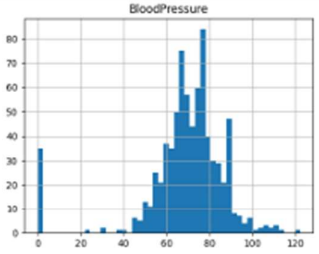


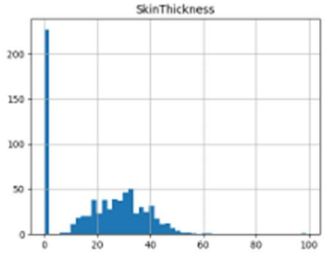


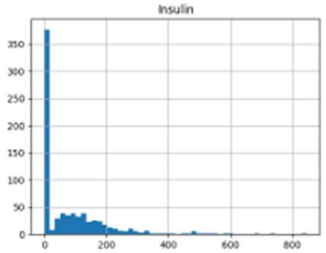


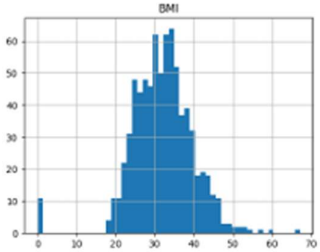


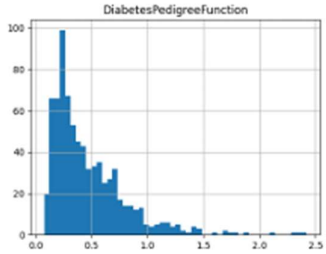


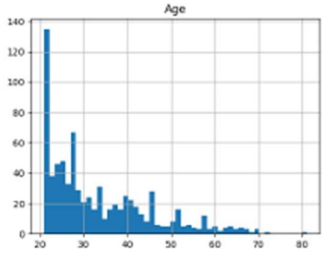


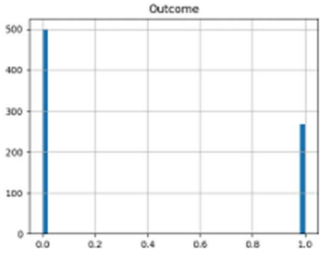














Result:

Thus , Data Collection and Initial Exploration was analysed and executed successfully

Experiment 16: Time Series Analysis of Website Traffic

Code:

```
import pandas as pd

import matplotlib.pyplot as plt

from statsmodels.tsa.seasonal import seasonal_decompose


data = pd.read_csv('website_traffic.csv', parse_dates=['Date'],
index_col='Date')


decomposition = seasonal_decompose(data['page_views'],
model='additive')

decomposition.plot()

plt.show()


data['page_views_Z'] = (data['page_views'] -
data['page_views'].mean()) / data['page_views'].std()


anomalies = data[abs(data['page_views_Z']) > 3]

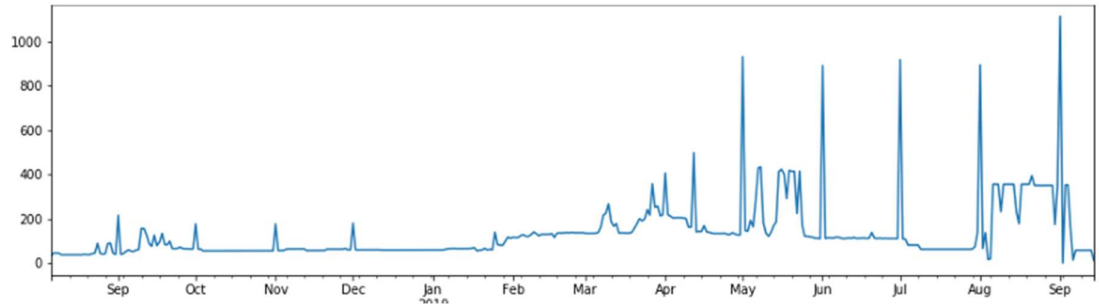

plt.plot(data.index, data['page_views'], label='page_views')

plt.scatter(anomalies.index, anomalies['page_views'], color='red',
label='Anomalies')

plt.legend()

plt.show()
```

Output:



Result:

Thus , Time Series Analysis of Website Traffic code was analysed and executed successfully.

Experiment 17:. Random Sampling and Sampling Distribution

Code:

```
import numpy as np
import matplotlib.pyplot as plt

population_mean = 50
population_std = 10
population_size = 100000
population = np.random.normal(population_mean, population_std, population_size)

sample_sizes = [30, 50, 100]
num_samples = 1000
sample_means = {}

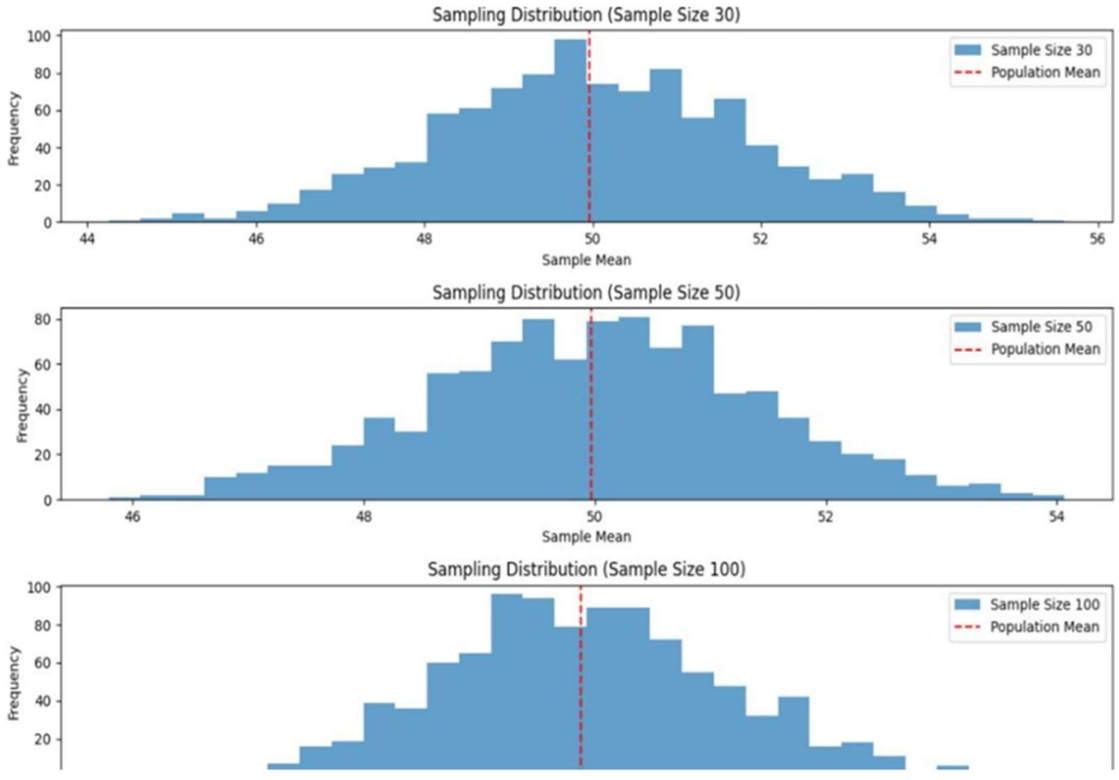
for size in sample_sizes:
    sample_means[size] = []
    for _ in range(num_samples):
        sample = np.random.choice(population, size=size, replace=False)
        sample_means[size].append(np.mean(sample))

plt.figure(figsize=(12, 8))
for i, size in enumerate(sample_sizes):
    plt.subplot(len(sample_sizes), 1, i + 1)
```

```
    plt.hist(sample_means[size], bins=30, alpha=0.7, label=f'Sample Size {size}')
    plt.title(f'Sampling Distribution (Sample size {size})')
    plt.xlabel('Sample Mean')
    plt.ylabel('Frequency')
    plt.legend()

plt.tight_layout()
plt.show()
```

Output:



Result:

Thus, Random Sampling and Sampling Distribution was analysed and executed successfully.

Experiment 18: Experiment to understand Matplotlib library use cases in Data Science through visualization

Code:

```
import pandas as pd

import numpy as np

import seaborn as sns

import matplotlib.pyplot as plt


data = pd.read_csv('/content/Dataset.csv')


data.info()

data.describe()

data.value_counts('variety')


sns.countplot(x='variety', data=data)

plt.show()


dummies = pd.get_dummies(data.variety)

FinalDataset = pd.concat([pd.get_dummies(data.variety), data.iloc[:, [0, 1, 2, 3]]], axis=1)

FinalDataset.head()


sns.pairplot(data, hue='variety', height=2.5)

plt.show()
```

```
sns.FacetGrid(data, hue='variety', height=5).map(sns.histplot,
'petallength(cm)').add_legend()

plt.show()


sns.FacetGrid(data, hue='variety', height=5).map(sns.histplot,
'PetalWidth(cm)').add_legend()

plt.show()


sns.FacetGrid(data, hue='variety', height=5).map(sns.histplot,
'SepalLength(cm)').add_legend()

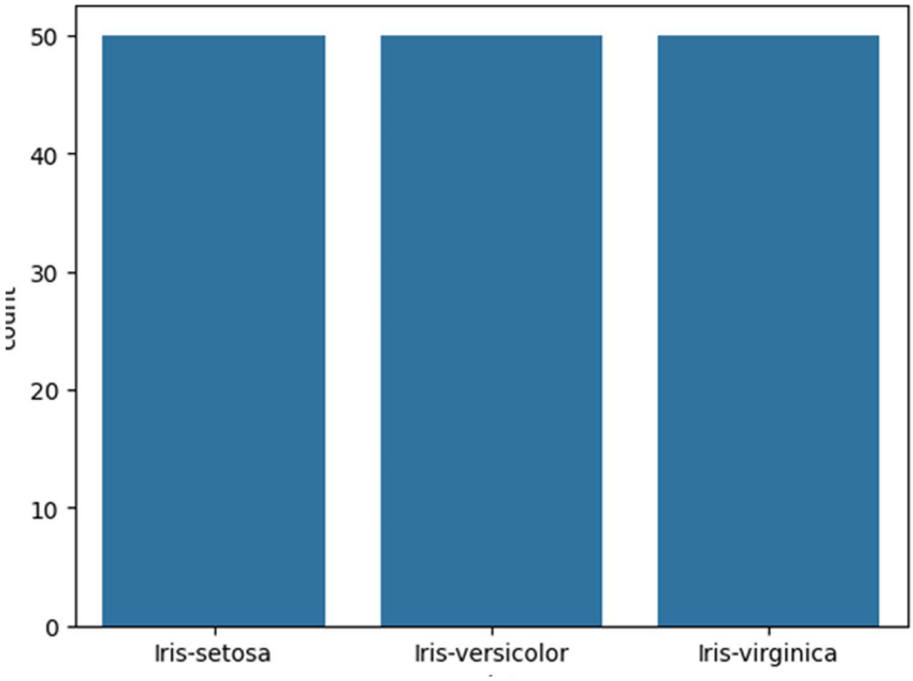
plt.show()

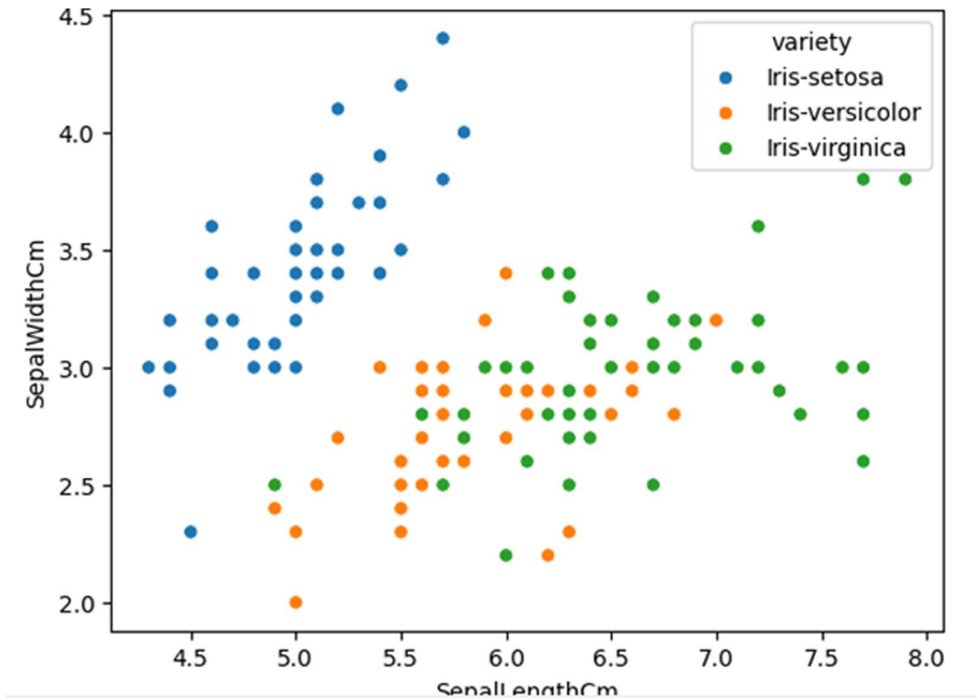

sns.FacetGrid(data, hue='variety', height=5).map(sns.histplot,
'SepalWidth(cm)').add_legend()

plt.show()
```

Output:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000





Result:

Thus , the Matplotlib library use cases in Data Science through visualization was analysed and executed successfully.

Exp No:19. Experiment to understand array function in Data science.

Code:

```
import numpy as np

array = np.random.randint(1, 100, 9)
a = np.sqrt(array)
print(array.ndim)

new_array = array.reshape(3, 3)
print(new_array.ndim)

new_array.ravel()
newm = new_array.reshape(3, 3)
print(newm[1:2, :3])

print(new_array[1:2, 1:2])
print(new_array[0:3, 0:0])
print(new_array[1:3])
```

Output:

```
  array([[83, 25, 19],
         [47, 62, 15],
         [96, 39, 51]])


_array.ndim

  2


_array.ravel()

  array([83, 25, 19, 47, 62, 15, 96, 39, 51]


ı=new_array.reshape(3,3)


ı

  array([[83, 25, 19],
         [47, 62, 15],
         [96, 39, 51]])


ı[2,1:3]

  array([39, 51])


ı[1:2,1:3]

  array([[62, 15]])


_array[0:3,0:0]

  array([], shape=(3, 0), dtype=int64)


_array[0:2,0:1]

  array([[83],
         [47]])

array[0:3,0:1]

 array([[83],
        [47],
        [96]])


array[1:3]

 array([[47, 62, 15],
        [96, 39, 51]])
```

Result:

Hence array function in Data science was understood and code executed successfully.

Exp No:20. Experiment to understand pandas library use cases in Data science.

Code:

```
import numpy as np

import pandas as pd


# Creating DataFrame from list[span_3](start_span)[span_3](end_span)

data_list = [[1, 'Smith', 50000], [2, 'John', 60000]]

df = pd.DataFrame(data_list)

df.columns = ['Empd', 'Name', 'Salary']


print(df.info())

print(df.head())

print(df.tail())


# Reading from CSV file[span_4](start_span)[span_4](end_span)

df = pd.read_csv('/content/employee.csv')


print(df.head())

print(df.tail())

print(df.info())


print(df['Salary'])
```

```
print(df['Salary'].var())[span_5](start_span)[span_5](end_span)
print(df['Salary'].std())[span_6](start_span)[span_6](end_span)
print(df.describe())[span_7](start_span)[span_7](end_span)
print(df.describe(include='all'))[span_8](start_span)[span_8](end_span)

emparray = df.values[span_9](start_span)[span_9](end_span)
employee_df = pd.DataFrame(emparray, columns=df.columns)[span_10](start_span)[span_10](end_span)
print(employee_df)[span_11](start_span)[span_11](end_span)
```

Output:

	R&D Spend	Administration	Marketing Spend	State	Profit
0	165349.20	136897.80	471784.10	New York	192261.83
1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99
4	142107.34	91391.77	366168.42	Florida	166187.94

	R&D Spend	Administration	Marketing Spend	State	Profit
45	1000.23	124153.04	1903.93	New York	64926.08
46	1315.46	115816.21	297114.46	Florida	49490.75
47	0.00	135426.92	0.00	California	42559.73
48	542.05	51743.15	0.00	New York	35673.41

	emp id	name	salary
0	1	SREE VARSSINI K S	5000
1	2	SREEMATHI B	6000
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH U G	8000

	emp id	name	salary
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH U G	8000
5	6	SRI HARSHAVARDHANAN R	3000

	salary
0	5000
1	6000
2	7000
3	5000
4	8000
5	3000
6	6000

	emp id	name	salary
count	7.000000	7	7.000000
unique	NaN	6	NaN
top	NaN	SRI HARSHAVARDHANAN R	NaN
freq	NaN	2	NaN
mean	4.000000	NaN	5714.285714
std	2.160247	NaN	1603.567451
min	1.000000	NaN	3000.000000
25%	2.500000	NaN	5000.000000
50%	4.000000	NaN	6000.000000
75%	5.500000	NaN	6500.000000

	emp id	name	salary
0	1	SREE VARSSINI K S	5000
1	2	SREEMATHI B	6000
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH U G	8000
5	6	SRI HARSHAVARDHANAN R	3000
6	7	SRI HARSHAVARDHANAN R	6000

Result:

Thus the pandas library use cases in data science was analysed and this code executed successfully.